# EXAM RESULT ANALYZER USING C++

**PROJECT TITLE:**

**EXAM RESULT ANALYZER USING C++**

**SUBJECT:**

**C++ Programming (Object Oriented Programming)**

**GROUP NUMBER:**

**Group No: 12**

**PROJECT MEMBERS:**


1. Antara Yogesh Palwankar. Roll No:150096725126
2. Atharva Ashok Dalvi. Roll No:150096725180
3. Saiyash Poojari. Roll No:150096725136
4. Aditya Kumbhar. Roll No: 150096725187


**ACADEMIC YEAR:**

2025 – 2026

**SUBMITTED TO:**

Prof: Anushree Ma’am.

# INTRODUCTION:

In the present education system, analyzing examination results manually can be time-consuming and error-prone. To overcome this problem, the **Exam Result Analyzer** project has been developed using the C++ programming language.

This project is a console-based application that helps in calculating students' total marks, percentage, and pass/fail status. It also provides overall class analysis such as number of students passed, failed, pass percentage, and identification of the top performer.

The project is designed using **Object Oriented Programming (OOP)** concepts, which makes the program structured, reusable, and easy to understand.

## AIM OF THE PROJECT:

The aim of this project is to develop a C++ program that efficiently analyzes student examination results by calculating percentages, determining pass or fail status, and performing overall class result analysis using object-oriented programming concepts.

## SCOPE OF THE PROJECT:

The scope of the Exam Result Analyzer project includes:

- Small-scale result analysis for schools and colleges
- Quick calculation of results with reduced manual effort
- Educational use for understanding OOP concepts
- A base model that can be enhanced with additional features such as grading systems, file storage, or graphical user interface in the future

## OBJECTIVES OF THE PROJECT:

The main objectives of the project are:

- To store marks of students in different subjects
- To calculate total marks and percentage
- To determine pass or fail status of each student
- To display individual student results
- To analyze overall class performance
- To identify the top-scoring student
- To implement object-oriented programming concepts in C++

## Program Environment & File Extensions Used:

This project is developed as a **console-based C++ application**.

### File Extensions Used:

- **.cpp** → Main source code file
- **.exe** → Executable file (after compilation)

### Compilation & Execution Options:

- Can be compiled using C++ compilers such as:
    - GCC
    - Code::Blocks
    - Turbo C++

The program runs directly in the **command line / terminal** environment.

## CONCEPTS USED:

The following C++ and Object Oriented Programming concepts are used in this project:

- Classes and Objects
- Constructors
- Encapsulation
- Arrays
- Member Functions
- Conditional Statements (if-else)
- Looping Statements (for loop)
- User Input and Output

These concepts help in organizing the program logically and efficiently.

## SYSTEM DESIGN / PROGRAM DESCRIPTION:

The project is divided into two main classes:

### 1. Student Class

The Student class is responsible for:

- Storing roll number and marks of students
- Calculating total marks and percentage
- Determining pass or fail result
- Displaying student details

### 2. ExamResultAnalyzer Class

The ExamResultAnalyzer class is responsible for:

- Managing multiple student objects
- Accepting data for all students
- Analyzing class performance
- Counting passed and failed students
- Identifying the top ranker

**This modular design improves readability and reusability of the code.**

## ALGORITHM: (Step-by-step)

1. Start the program
2. Accept the number of students
3. For each student:
    - Input marks for all subjects
    - Calculate total marks
    - Calculate percentage
    - Determine pass or fail status
4. Display individual student results
5. Calculate total number of passed and failed students
6. Calculate overall pass percentage
7. Identify the highest percentage (top ranker)
8. Display class analysis
9. End the program

## FLOWCHART:

The flowchart of the project follows these steps:

- Start
- Input number of students
- Input marks for each student
- Calculate total and percentage
- Determine result
- Analyze class performance
- Display output
- End

# ADVANTAGES OF THE PROJECT:

- Reduces manual effort in result calculation
- Minimizes human errors
- Easy to understand and use
- Provides quick class analysis
- Demonstrates effective use of OOP concepts
- Can be extended for advanced features

# SCREENSHOTS:

## CODE

operoverloading.cpp | project.cpp | file.cpp | abc.txt | 2.cpp | ps2.cpp

mini project > project.cpp > Student > inputStudent(int)

```cpp
class Student {
    void calculateResult() {
            result = "Fail";
    }
    float getPercentage() {
        return percentage;
    }
    string getResult() {
        return result;
    }
    void displayStudent() {
        cout << "Roll No: " << roll
             << " | Percentage: " << percentage
             << " | Result: " << result << endl;
    }
};
class ExamResultAnalyzer {
private:
    Student s[50];
    int n;
public:
    ExamResultAnalyzer(int count) {
        n = count;
    }
    void inputAllStudents() {
        for (int i = 0; i < n; i++) {
            s[i].inputStudent(i + 1);
        }
    }
    void analyzeResults() {
        int passCount = 0;
        int failCount = 0;
        float highest = 0;
        for (int i = 0; i < n; i++) {
            if (s[i].getResult() == "Pass")
                passCount++;
            else
                failCount++;
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
antarapalwankar@Antaras-MacBook-Air C++ %
```

Ln 18, Col 67 Spaces: 4 UTF-8 LF C++ Finish Setup Go Live Mac

operoverloading.cpp | project.cpp | file.cpp | abc.txt | 2.cpp | ps2.cpp

mini project > project.cpp > Student > inputStudent(int)

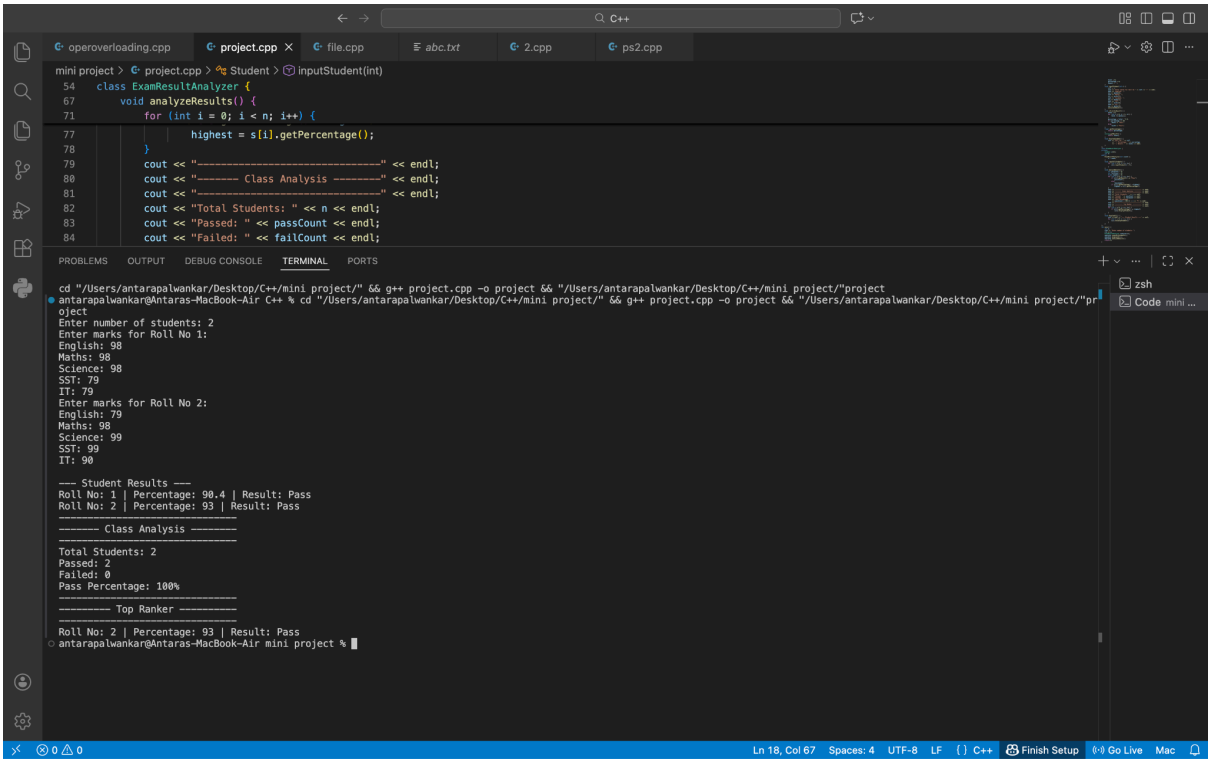
```cpp
class ExamResultAnalyzer {
    void analyzeResults() {
        for (int i = 0; i < n; i++) {
            highest = s[i].getPercentage();
        }
        cout << "------------------------------" << endl;
        cout << "------- Class Analysis --------" << endl;
        cout << "------------------------------" << endl;
        cout << "Total Students: " << n << endl;
        cout << "Passed: " << passCount << endl;
        cout << "Failed: " << failCount << endl;
        cout << "Pass Percentage: "
             << (passCount * 100.0) / n << "%" << endl;
        cout << "------------------------------" << endl;
        cout << "--------- Top Ranker ----------" << endl;
        cout << "------------------------------" << endl;
        for (int i = 0; i < n; i++) {
            if (s[i].getPercentage() == highest)
                s[i].displayStudent();
        }
    }
    void displayAll() {
        cout << endl << "--- Student Results ---" << endl;
        for (int i = 0; i < n; i++) {
            s[i].displayStudent();
        }
    }
};
int main() {
    int n;
    cout << "Enter number of students: ";
    cin >> n;
    ExamResultAnalyzer analyzer(n);
    analyzer.inputAllStudents();
    analyzer.displayAll();
    analyzer.analyzeResults();
    return 0;
}
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
antarapalwankar@Antaras-MacBook-Air C++ %
```

Ln 18, Col 67 Spaces: 4 UTF-8 LF C++ Finish Setup Go Live Mac

## OUTPUT:




## CONCLUSION:

The Exam Result Analyzer project successfully demonstrates the use of C++ and object-oriented programming concepts to analyze student examination results. It efficiently calculates percentages, determines pass or fail status, and provides overall class performance analysis. The project is simple, accurate, and useful for basic result processing, and it can be further enhanced with additional features in the future.

**THANK YOU**